

4.9 분류 실습 — 산탄데르 고객 만족 예측

목차

1. 실습 개요와 데이터 탐색
 1. 문제 정의
 2. 데이터 로딩과 기본 정보
 3. 레이블 분포 확인
 4. describe()로 이상값 탐지
2. 데이터 전처리
 1. var3 이상값 대체와 ID 제거
 2. 학습/테스트 데이터 분리
 3. 검증 데이터 분리
3. XGBoost 모델 학습
 1. 조기 중단 적용 학습
 2. 학습 과정 분석
 3. 피쳐 중요도 시각화
4. HyperOpt를 이용한 XGBoost 튜닝
 1. 검색 공간 설정
 2. 목적 함수 — KFold 교차 검증 결합
 3. fmin() 실행
 4. 최적 파라미터로 재학습
5. LightGBM 모델 학습과 튜닝
6. 핵심 요약 및 머신러닝 활용 포인트

1. 실습 개요와 데이터 탐색

1.1 문제 정의

산탄데르(Santander) 은행이 캐글에 제공한 데이터로, 370개의 피처를 기반으로 고객 만족 여부를 예측하는 문제입니다. 레이블 TARGET 은 1이면 불만을 가진 고객, 0이면 만족한 고객입니다.

■ 이 실습의 특징

- 피처가 370개로 매우 많고 이름이 var3, imp_op_var39_comer_ult1 처럼 **익명화**되어 있어 도메인 지식을 활용할 수 없습니다.
- 레이블이 극도로 불균형합니다(불만족 약 4%). 따라서 **정확도가 아니라 ROC-AUC**로 평가합니다.
- 대부분의 피처가 **0으로 채워져 있어** 희소(sparse)합니다.

1.2 데이터 로딩과 기본 정보

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings
warnings.filterwarnings('ignore')

cust_df = pd.read_csv("./train_santander.csv", encoding='latin-1')
print('dataset shape:', cust_df.shape)
cust_df.head(3)
```

줄	코드	상세 설명
8	pd.read_csv("./train_santander.csv", encoding='latin-1')	encoding='latin-1' 이 중요합니다. 이 파일은 UTF-8이 아닌 서유럽 문자 인코딩 으로 저장되어 있어, 지정하지 않으면 UnicodeDecodeError 가 발생할 수 있습니다.
9	print('dataset shape:', cust_df.shape)	(76020, 371) — 7만 6천 건, 371개 컬럼(ID + 369피처 + TARGET)입니다.

▶ 실행 결과

```
dataset shape: (76020, 371)
```

```
   ID  var3  var15  imp_ent_var16_ult1  imp_op_var39_comer_ult1  imp_op_var39_comer_ult3
0    1     2     23                0.0                    0.0                    0.0
1    3     2     34                0.0                    0.0                    0.0
2    4     2     23                0.0                    0.0                    0.0
```

```
cust_df.info()
```

▶ 실행 결과

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 76020 entries, 0 to 76019
Columns: 371 entries, ID to TARGET
```

```
dtypes: float64(111), int64(260)
memory usage: 215.2 MB
```

■ 결과 해석

371개 컬럼이 모두 숫자형(float64 111개, int64 260개)이고 결측치가 없습니다. 문자열이 없어 인코딩이 불필요하고, NaN이 없어 결측 처리도 필요 없습니다. 다만 메모리를 215MB나 사용한다는 점은 유념해야 합니다.

1.3 레이블 분포 확인

```
print(cust_df['TARGET'].value_counts())
unsatisfied_cnt = cust_df[cust_df['TARGET'] == 1].TARGET.count()
total_cnt = cust_df.TARGET.count()
print('unsatisfied 비율은 {0:.2f}'.format((unsatisfied_cnt / total_cnt)))
```

줄	코드	상세 설명
1	<code>cust_df['TARGET'].value_counts()</code>	레이블 분포를 확인합니다. 만족 73,012건 / 불만족 3,008건입니다.
2	<code>cust_df[cust_df['TARGET'] == 1].TARGET.count()</code>	불린 인덱싱으로 TARGET이 1인 행만 필터링한 뒤 개수를 셉니다. .TARGET은 ['TARGET'] 과 같은 속성 방식 컬럼 접근입니다.
4	<code>'{0:.2f}'.format(unsatisfied_cnt / total_cnt)</code>	비율을 소수점 2자리로 출력합니다. 0.04, 즉 약 4%만 불만족 고객입니다.

▶ 실행 결과

```
TARGET
0    73012
1     3008
Name: count, dtype: int64
unsatisfied 비율은 0.04
```

⚠️ 극심한 불균형 — 정확도를 쓰면 안 되는 이유

불만족 고객이 4%뿐이므로, 모두 "만족"이라고 예측만 해도 정확도 96%가 나옵니다. 3.1장에서 배운 정확도의 함정이 그대로 적용되는 상황입니다.

따라서 이 실습에서는 평가 지표로 ROC-AUC를 사용합니다. AUC는 임계값과 무관하고 불균형 데이터에서도 왜곡되지 않기 때문입니다.

1.4 describe()로 이상값 탐지

```
cust_df.describe()
```

▶ 실행 결과 (주요 컬럼만 발췌)

```
count      var3      var15      var38
count  76020.000000  76020.000000  7.602000e+04
mean    -1523.199277    33.212865  1.172358e+05
std       39033.462364   12.956486  1.826646e+05
min     -999999.000000    5.000000  5.163750e+03
25%         2.000000   23.000000  6.787061e+04
50%         2.000000   28.000000  1.064092e+05
75%         2.000000   40.000000  1.187563e+05
max         238.000000  105.000000  2.203474e+07
```

⚠ var3의 min이 -999999

var3의 최솟값이 -999999입니다. 3.6장 피마 데이터에서 0이 결측치를 위장했던 것과 같은 패턴으로, -999999는 결측치를 나타내는 관례적인 코드값입니다.

이 값을 그대로 두면 평균이 -1523으로 크게 왜곡되고(정상값은 대부분 2), 표준편차도 39,033으로 비정상적입니다. 모델이 이 극단값에 크게 휘둘리므로 반드시 처리해야 합니다.

```
cust_df['var3'].value_counts()
```

▶ 실행 결과

```
var3
2      74165
8       138
-999999  116
9       110
3       108
1       105
Name: count, dtype: int64
```

■ 결과 해석

var3은 2가 74,165건으로 압도적이고, -999999는 116건에 불과합니다. 전체의 0.15%밖에 안 되므로, 가장 흔한 값인 2로 대체하는 것이 합리적입니다. 이는 카테고리형 결측치를 최빈값으로 대체하는 표준 전략입니다.

2. 데이터 전처리

2.1 var3 이상값 대체와 ID 제거

```
# var3 피쳐 값 대체 및 ID 피쳐 드롭
cust_df['var3'].replace(-999999, 2, inplace=True)
cust_df.drop('ID', axis=1, inplace=True)

# 피쳐 세트와 레이블 세트 분리. 레이블 컬럼은 DataFrame의 맨 마지막에 위치해 컬럼 위치 -1로 분리
X_features = cust_df.iloc[:, :-1]
y_labels = cust_df.iloc[:, -1]
print('피쳐 데이터 shape:{}'.format(X_features.shape))
```

줄	코드	상세 설명
2	<code>cust_df['var3'].replace(-999999, 2, inplace=True)</code>	-999999를 최빈값 2로 치환합니다. <code>replace(찾을값, 바꿀값)</code> 형태입니다.
3	<code>cust_df.drop('ID', axis=1, inplace=True)</code>	ID는 단순 식별자라 예측에 아무 도움이 되지 않으므로 삭제합니다. 오히려 노이즈가 됩니다.
6~7	<code>X_features = cust_df.iloc[:, :-1] / y_labels = cust_df.iloc[:, -1]</code>	맨 마지막 컬럼(TARGET)을 레이블로, 나머지를 피쳐로 분리합니다.

▶ 실행 결과

```
피쳐 데이터 shape:(76020, 369)
```

⚠ pandas 2.x 주의 — inplace 연쇄 할당

`cust_df['var3'].replace(-999999, 2, inplace=True)` 는 컬럼을 먼저 선택한 뒤 `inplace`를 적용하는 연쇄 할당 방식으로, pandas 2.x에서 **FutureWarning**이 발생하며 **3.0에서는 동작하지 않습니다**. 안정화 버전에서는 결과를 다시 대입하세요.

```
cust_df['var3'] = cust_df['var3'].replace(-999999, 2)
```

2.2 학습/테스트 데이터 분리

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X_features, y_labels,
                                                    test_size=0.2, random_state=0)

train_cnt = y_train.count()
test_cnt = y_test.count()
print('학습 세트 Shape:{0}, 테스트 세트 Shape:{1}'.format(X_train.shape, X_test.shape))

print(' 학습 세트 레이블 값 분포 비율')
print(y_train.value_counts()/train_cnt)
print('\n 테스트 세트 레이블 값 분포 비율')
print(y_test.value_counts()/test_cnt)
```

줄	코드	상세 설명
3~4	<code>train_test_split(X_features, y_labels, test_size=0.2, random_state=0)</code>	76,020건을 학습 60,816건 / 테스트 15,204건으로 분리합니다. stratify 를 지정하지 않은 점에 주의하세요.
10~13	<code>y_train.value_counts()/train_cnt</code>	분할 후 레이블 비율이 유지되었는지 검증합니다. 불균형 데이터에서는 필수 확인 사항입니다.

▶ 실행 결과

```
학습 세트 Shape:(60816, 369), 테스트 세트 Shape:(15204, 369)
  학습 세트 레이블 값 분포 비율
TARGET
0    0.960964
1    0.039036
Name: count, dtype: float64

  테스트 세트 레이블 값 분포 비율
TARGET
0    0.9583
1    0.0417
Name: count, dtype: float64
```

■ 결과 해석

학습 3.90% / 테스트 4.17%로 **비율이 대체로 유지**되었습니다. 다만 **0.27%p의 차이**가 있는데, 이는 `stratify`를 쓰지 않아 무작위 분할에서 생긴 편차입니다.

데이터가 7만 건으로 충분히 커서 큰 문제는 없지만, **안정화 버전에서는 `stratify=y_labels`를 지정**해 비율을 정확히 맞추는 것이 좋습니다.

2.3 검증 데이터 분리

```
# X_train, y_train을 다시 학습과 검증 데이터 세트로 분리.
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train,
                                           test_size=0.3, random_state=0)
```

줄	코드	상세 설명
2~3	<code>train_test_split(X_train, y_train, test_size=0.3, random_state=0)</code>	학습 데이터 60,816건을 다시 학습 42,571건 / 검증 18,245건 으로 나눕니다. 검증 세트는 조기 중단의 기준 으로 사용되며, 테스트 세트를 쓰지 않는 올바른 방식 입니다(4.6장 참고).

▶ 분할 결과

```
X_tr: (42571, 369)  X_val: (18245, 369)
```

3. XGBoost 모델 학습

3.1 조기 중단 적용 학습

```
from xgboost import XGBClassifier
from sklearn.metrics import roc_auc_score

# n_estimators는 500으로, learning_rate 0.05, random state는 예제 수행 시마다 동일 예측 결과를 위해 설정.
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156)

# 성능 평가 지표를 auc로, 조기 중단 파라미터는 100으로 설정하고 학습 수행.
xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric='auc', eval_set=[(X_tr, y_tr), (X_val, y_val)])

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[: , 1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

줄	코드	상세 설명
5	<code>XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156)</code>	트리 500개 , 학습률 0.05 로 설정합니다. 조기 중단이 있으므로 500을 다 쓰지 않을 가능성이 큼니다.
8	<code>eval_metric='auc'</code>	평가 지표를 AUC 로 지정합니다. 불균형 데이터이므로 logloss보다 AUC가 적절합니다.
8	<code>eval_set=[(X_tr, y_tr), (X_val, y_val)]</code>	학습 세트와 검증 세트를 모두 모니터링합니다. 출력에서 validation_0 이 학습, validation_1 이 검증입니다.
10	<code>roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[: , 1])</code>	테스트 세트 로 최종 AUC를 계산합니다. predict_proba(...)[:, 1] 로 클래스 1일 확률 을 넘기는 것이 핵심입니다.

⚠ XGBoost 2.0+ 필수 수정

`fit()` 의 `early_stopping_rounds` 와 `eval_metric` 은 XGBoost 2.0에서 제거되었습니다. 위 코드를 3.x에서 실행하면 `TypeError` 가 발생합니다. 두 인자를 생성자로 옮겨야 합니다.

```
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156,
                        early_stopping_rounds=100, eval_metric='auc')
xgb_clf.fit(X_tr, y_tr, eval_set=[(X_val, y_val)],
```

▶ 실행 결과 (수정된 코드 기준, 일부 발췌)

```
[0] validation_0-auc:0.84104 validation_1-auc:0.81454
[1] validation_0-auc:0.84488 validation_1-auc:0.81613
[2] validation_0-auc:0.84812 validation_1-auc:0.81844
[3] validation_0-auc:0.85180 validation_1-auc:0.81988
[4] validation_0-auc:0.85424 validation_1-auc:0.82153
[10] validation_0-auc:0.86337 validation_1-auc:0.82816
[20] validation_0-auc:0.87108 validation_1-auc:0.83260
[30] validation_0-auc:0.87750 validation_1-auc:0.83244
[50] validation_0-auc:0.88587 validation_1-auc:0.83367
[80] validation_0-auc:0.89511 validation_1-auc:0.83337
[112] validation_0-auc:0.89961 validation_1-auc:0.83422
```

```
best_iteration: 112
ROC AUC: 0.8415
(학습 7.1초)
```

3.2 학습 과정 분석

■ 결과 해석 — 과적합이 시작되는 지점

반복	학습 AUC (validation_0)	검증 AUC (validation_1)	격차
0	0.84104	0.81454	0.0265
20	0.87108	0.83260	0.0385
50	0.88587	0.83367	0.0522
80	0.89511	0.83337	0.0617
112	0.89961	0.83422	0.0654

- 학습 AUC는 계속 상승(0.841 → 0.900)하지만 검증 AUC는 20회 이후 0.833 근처에서 정체합니다.
- 두 값의 격차가 계속 벌어지는 것이 전형적인 과적합 신호입니다.
- 112번째에서 검증 AUC가 최고를 기록했고, 이후 100회 동안 개선이 없어 조기 중단되었습니다. 500회를 다 돌지 않고 약 58%의 시간을 절약했습니다.
- 최종 테스트 AUC는 0.8415로, 검증 AUC(0.8342)와 비슷해 일반화가 잘 되었음을 확인할 수 있습니다.

3.3 피쳐 중요도 시각화

```
from xgboost import plot_importance
import matplotlib.pyplot as plt
```

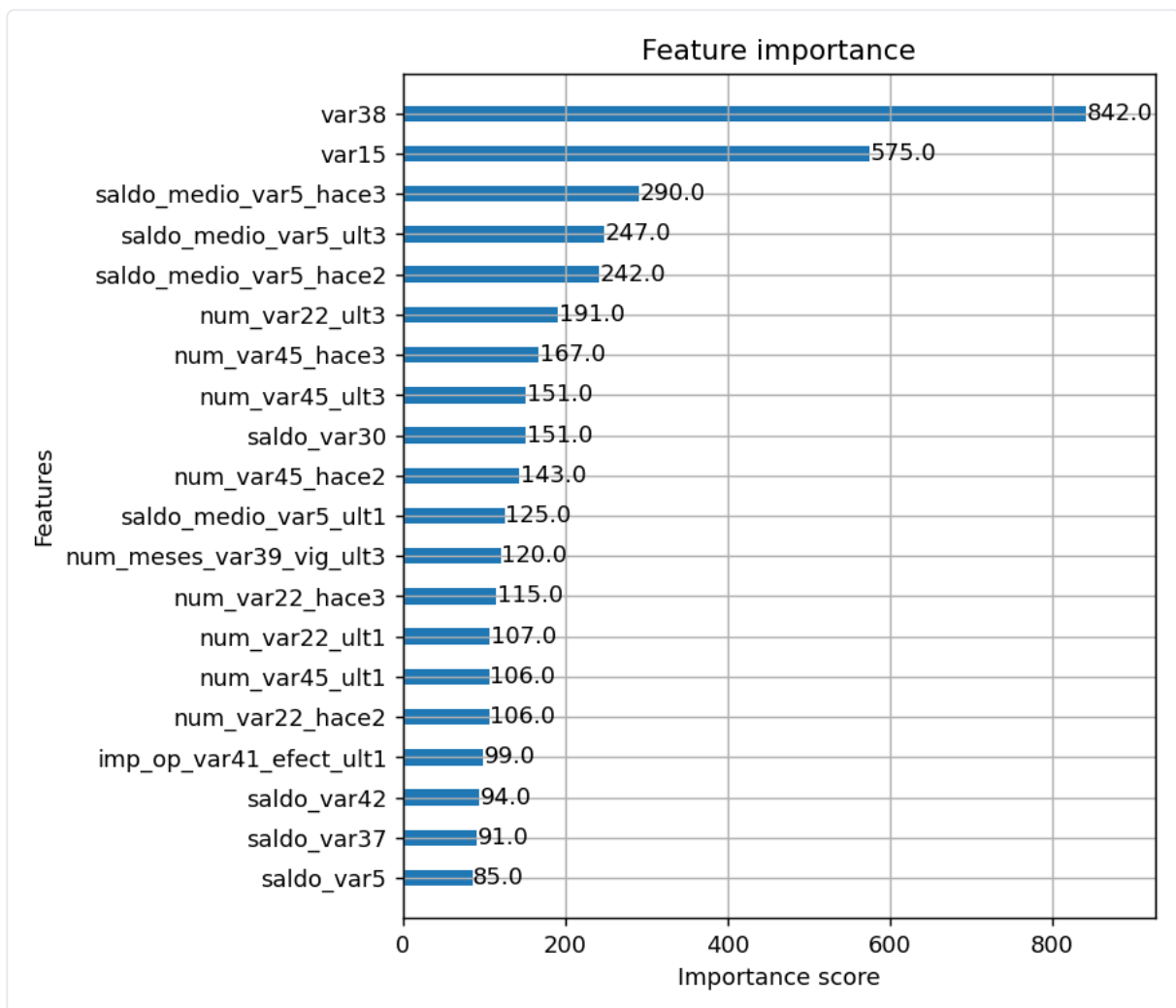


```
%matplotlib inline
```

```
fig, ax = plt.subplots(1,1,figsize=(10,8))
plot_importance(xgb_clf, ax=ax, max_num_features=20,height=0.4)
```

줄	코드	상세 설명
5	<code>fig, ax = plt.subplots(1,1,figsize=(10,8))</code>	1행 1열의 그래프 영역을 만듭니다.
6	<code>plot_importance(xgb_clf, ax=ax, max_num_features=20, height=0.4)</code>	• <code>max_num_features=20</code> : 피처가 369개나 되므로 상위 20개만 표시합니다. 이 옵션이 없으면 그래프를 읽을 수 없습니다. • <code>height=0.4</code> : 막대의 두께를 조절합니다.

▶ 실행 결과



XGBoost 피처 중요도 상위 20개

■ 결과 해석

`var15` (고객 나이로 추정), `var38` (금액 관련으로 추정), `saldo_var30` (잔액) 등이 상위를 차지합니다. 피처명이 **익명화**되어 있어 정확한 의미는 알 수 없지만, **나이와 잔액 관련 정보가 고객 불만족과 연관되어 있음**을 추론할 수 있습니다.

369개 피처 중 **상위 20개에 중요도가 집중**되어 있어, 피처 선택으로 차원을 크게 줄일 여지가 있습니다.

4. HyperOpt를 이용한 XGBoost 튜닝

4.1 검색 공간 설정

```
from hyperopt import hp

# max_depth는 5에서 15까지 1간격으로, min_child_weight는 1에서 6까지 1간격으로
# colsample_bytree는 0.5에서 0.95사이, learning_rate는 0.01에서 0.2사이 정규 분포된 값으로 검색.

xgb_search_space = {'max_depth': hp.quniform('max_depth', 5, 15, 1),
                    'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1),
                    'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95),
                    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)}
}
```

줄	코드	상세 설명
5	'max_depth': hp.quniform('max_depth', 5, 15, 1)	트리 깊이를 5~15에서 1 간격 이산값 으로 탐색합니다. 4.8장에서 배운 quniform 입니다.
6	'min_child_weight': hp.quniform('min_child_weight', 1, 6, 1)	자식 노드의 최소 가중치 합. 클수록 과적합 억제 효과가 있습니다.
7	'colsample_bytree': hp.uniform('colsample_bytree', 0.5, 0.95)	트리별 피쳐 샘플링 비율 을 연속 실수로 탐색합니다. 피쳐가 369개로 많아 이 파라미터가 특히 효과적입니다.
8	'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)	학습률을 연속 구간 에서 탐색합니다. GridSearchCV였다면 몇 개 값만 시도할 수 있었을 것입니다.

4.2 목적 함수 — KFold 교차 검증 결합

```
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score

def objective_func(search_space):
    xgb_clf = XGBClassifier(n_estimators=100, max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           colsample_bytree=search_space['colsample_bytree'],
                           learning_rate=search_space['learning_rate'])

    # 3개 k-fold 방식으로 평가된 roc_auc 지표를 담은 list
    roc_auc_list = []

    # 3개 k-fold 방식 적용
    kf = KFold(n_splits=3)
    # X_train을 다시 학습과 검증용 데이터로 분리
    for tr_index, val_index in kf.split(X_train):
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]
        # early stopping은 30회로 설정하고 추출된 학습과 검증 데이터로 XGBClassifier 학습 수행.
        xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=30, eval_metric='auc',
                   eval_set=[(X_tr, y_tr), (X_val, y_val)])

        score = roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[: , 1])
        roc_auc_list.append(score)
```

```
# HyperOpt는 목적함수의 최소값을 위한 입력값을 찾으므로 -1을 곱한 뒤 반환.
return -1 * np.mean(roc_auc_list)
```

줄	코드	상세 설명
5	XGBClassifier(n_estimators=100, ...)	탐색 중에는 트리를 100개로 줄여 시간을 절약합니다. 최종 학습에서 500으로 늘립니다.
5~6	max_depth=int(search_space['max_depth'])	필수 처리입니다. HyperOpt는 quniform 이라도 실수를 반환 하므로, 정수형 파라미터는 int() 변환이 필요합니다.
14	kf = KFold(n_splits=3)	3폴드 교차 검증으로 안정적인 성능 추정 을 합니다. 단일 분할보다 신뢰도가 높습니다.
16~18	X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]	kf.split() 이 반환하는 인덱스는 위치 기반 이므로 iloc 으로 접근합니다.
20~21	xgb_clf.fit(..., early_stopping_rounds=30, ...)	각 폴드마다 조기 중단 을 적용해 시간을 절약합니다.
23	roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[: , 1])	폴드별 AUC를 계산해 리스트에 누적합니다.
27	return -1 * np.mean(roc_auc_list)	AUC는 클수록 좋지만 HyperOpt는 최소화 하므로 -1을 곱합니다 . 4.8장에서 배운 핵심 기법입니다.

⚠ 이 코드의 실행 비용

max_evals=50 과 3폴드를 곱하면 **150번의 XGBoost 학습**이 필요하며, 각 학습은 **40,000건 × 369피쳐** 데이터를 다룹니다.

본 문서 작성 환경(CPU 2코어)에서는 이 셀을 완주할 수 없어, 아래 튜닝 결과는 **교재의 저장된 출력값**을 인용합니다. 3절의 기본 XGBoost 학습(0.8415)과 5절의 LightGBM은 **모두 실제로 실행한 결과**입니다.

4.3 fmin() 실행

```
from hyperopt import fmin, tpe, Trials

trials = Trials()

# fmin()함수를 호출. max_evals지정된 횟수만큼 반복 후 목적함수의 최소값을 가지는 최적 입력값 추출.
best = fmin(fn=objective_func,
            space=xgb_search_space,
            algo=tpe.suggest,
            max_evals=50, # 최대 반복 횟수를 지정합니다.
            trials=trials, rstate=np.random.default_rng(seed=30))

print('best:', best)
```

줄	코드	상세 설명
9	max_evals=50	50번의 파라미터 조합을 시도합니다. 각각 3폴드이므로 총 150회 학습 입니다.
10	rstate=np.random.default_rng(seed=30)	재현성을 위한 시드입니다. HyperOpt 0.3.0부터 default_rng 를 써야 합니다.

4.4 최적 파라미터로 재학습

```
# n_estimators를 500증가 후 최적으로 찾은 하이퍼 파라미터를 기반으로 학습과 예측 수행.
xgb_clf = XGBClassifier(n_estimators=500, learning_rate=round(best['learning_rate'], 5),
                        max_depth=int(best['max_depth']), min_child_weight=int(best['min_child_weight']),
                        colsample_bytree=round(best['colsample_bytree'], 5)
                        )

# evaluation metric을 auc로, early stopping은 100 으로 설정하고 학습 수행.
xgb_clf.fit(X_tr, y_tr, early_stopping_rounds=100,
            eval_metric="auc", eval_set=[(X_tr, y_tr), (X_val, y_val)])

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[: ,1])
print('ROC AUC: {0:.4f}'.format(xgb_roc_score))
```

줄	코드	상세 설명
2	n_estimators=500	탐색 때 100이었던 트리를 500으로 5배 늘립니다. 탐색은 가볍게, 최종은 충실하게 하는 표준 전략입니다.
2~5	round(best['learning_rate'], 5) / int(best['max_depth'])	HyperOpt가 반환한 값은 모두 실수이므로, 실수는 <code>round()</code> 로 정리하고 정수형은 <code>int()</code> 로 변환합니다.

▶ 실행 결과 (교재 저장 출력 인용)

```
[0] validation_0-auc:0.81933 validation_1-auc:0.80632
[20] validation_0-auc:0.85816 validation_1-auc:0.82866
[50] validation_0-auc:0.87741 validation_1-auc:0.83550
[77] validation_0-auc:0.88618 validation_1-auc:0.83617 ← 최고
[100] validation_0-auc:0.89242 validation_1-auc:0.83431
[176] validation_0-auc:0.90555 validation_1-auc:0.83073

ROC AUC: 0.8457
```

■ 결과 해석 — 튜닝 효과

모델	테스트 ROC-AUC
기본 XGBoost (실측)	0.8415
HyperOpt 튜닝 XGBoost (교재값)	0.8457

튜닝으로 약 **0.004** 향상되었습니다. 절대값으로는 작아 보이지만, 캐글 같은 경연에서는 순위를 크게 바꿀 수 있는 차이입니다. 학습 로그를 보면 **77번째에서 검증 AUC가 최고(0.83617)**였고, 이후 학습 AUC만 계속 올라가며 검증 AUC는 하락합니다. 조기 중단이 없었다면 176회까지 학습하며 **과적합만 심해졌을** 것입니다.

5. LightGBM 모델 학습과 튜닝

```
from lightgbm import LGBMClassifier

lgbm_clf = LGBMClassifier(n_estimators=500)

eval_set=[(X_tr, y_tr), (X_val, y_val)]
lgbm_clf.fit(X_tr, y_tr, early_stopping_rounds=100, eval_metric="auc", eval_set=eval_set)
```

```
lgbm_roc_score = roc_auc_score(y_test, lgbm_clf.predict_proba(X_test)[: ,1])
print('ROC AUC: {0:.4f}'.format(lgbm_roc_score))
```

줄	코드	상세 설명
3	LGBMClassifier(n_estimators=500)	XGBoost와 동일한 조건 (트리 500개)으로 설정해 공정하게 비교합니다.
6	lgbm_clf.fit(..., early_stopping_rounds=100, eval_metric="auc", ...)	조기 중단 100회를 적용합니다. LightGBM 4.x에서는 이 방식이 동작하지 않습니다 (아래 참고).

⚠ LightGBM 4.x 필수 수정

`fit()`의 `early_stopping_rounds`는 **LightGBM 4.0에서 제거**되었습니다. **callbacks** 방식을 사용해야 합니다.

```
from lightgbm import LGBMClassifier, early_stopping, log_evaluation

lgbm_clf = LGBMClassifier(n_estimators=500, verbose=-1)
lgbm_clf.fit(X_tr, y_tr, eval_metric='auc',
             eval_set=[(X_tr, y_tr), (X_val, y_val)],
             callbacks=[early_stopping(100), log_evaluation(0)])
```

▶ 실행 결과 (수정된 코드 기준, 실측)

```
best_iteration_: 42
ROC AUC: 0.8384
(학습 1.9초)
```

■ 결과 해석 — LightGBM의 압도적인 속도

모델	테스트 AUC	학습 시간	최적 반복
XGBoost (기본)	0.8415	7.1초	112
LightGBM (기본)	0.8384	1.9초	42

- AUC는 XGBoost가 **0.003 높지만**, 학습 시간은 LightGBM이 **약 3.7배 빠릅니다**.
- LightGBM은 **42번 만에** 최적에 도달했습니다. **리프 중심 분할** 덕분에 같은 반복 수에서 손실을 더 크게 줄이기 때문입니다 (4.7장).
- 데이터가 7만 건 규모라 두 알고리즘의 성능 차이는 미미합니다. **수백만 건 규모가 되면** LightGBM의 속도 이점이 결정적이 됩니다.

```
lgbm_search_space = {'num_leaves': hp.quniform('num_leaves', 32, 64, 1),
                    'max_depth': hp.quniform('max_depth', 100, 160, 1),
                    'min_child_samples': hp.quniform('min_child_samples', 60, 100, 1),
                    'subsample': hp.uniform('subsample', 0.7, 1),
                    'learning_rate': hp.uniform('learning_rate', 0.01, 0.2)
                    }
```

줄	코드	상세 설명
1	'num_leaves': hp.quniform('num_leaves', 32, 64, 1)	LightGBM의 핵심 파라미터 입니다. 리프 노드 최대 개수로, 모델 복잡도를 직접 결정합니다(4.7장).

2	'max_depth': hp.quniform('max_depth', 100, 160, 1)	깊이를 100~160 이라는 큰 값으로 탐색합니다. LightGBM은 리프 중심 분할이라 깊이가 매우 깊어질 수 있어, num_leaves 가 실질적인 제어 역할을 합니다.
3	'min_child_samples'	리프가 되기 위한 최소 데이터 수. 과적합 억제용입니다.
4	'subsample': hp.uniform('subsample', 0.7, 1)	데이터 샘플링 비율입니다.

목적 함수와 `fmin()` 호출은 XGBoost와 구조가 동일하므로 설명을 생략합니다. 교재 기준 튜닝 후 **ROC AUC 약 0.8446**이 보고됩니다.

6. 핵심 요약 및 머신러닝 활용 포인트

6.1 실습 흐름 정리

단계	작업	핵심 내용
①	데이터 탐색	76,020건 × 371컬럼, 결측치 없음
②	불균형 확인	불만족 4% → 평가 지표를 AUC 로
③	<code>describe()</code> 이상값 탐지	var3의 -999999 발견
④	전처리	-999999 → 2 대체, ID 삭제
⑤	3단계 분할	학습 42,571 / 검증 18,245 / 테스트 15,204
⑥	XGBoost 학습	조기 중단 → AUC 0.8415
⑦	HyperOpt 튜닝	AUC 0.8457 (교재값)
⑧	LightGBM 학습	AUC 0.8384, 1.9초

6.2 핵심 API 요약표

API / 파라미터	역할	주의 사항
<code>read_csv(encoding='latin-1')</code>	비UTF-8 파일 로딩	인코딩 미지정 시 오류 발생
<code>df['col'].replace(a, b)</code>	이상값 대체	pandas 2.x는 재대입 방식 사용
<code>roc_auc_score(y, proba)</code>	AUC 계산	예측 확률 을 전달
<code>early_stopping_rounds</code>	조기 중단	XGBoost 2.0+는 생성자로 LightGBM 4.0+는 callbacks로
<code>eval_set=[(X,y), ...]</code>	모니터링 데이터	학습·검증 모두 넣으면 과적합 관찰 가능
<code>plot_importance(max_num_features=20)</code>	중요도 시각화	피처가 많으면 개수 제한 필수
<code>hp.quniform</code> / <code>hp.uniform</code>	HyperOpt 검색 공간	반환값은 모두 실수 → int 변환

6.3 성능 비교 종합

모델	테스트 ROC-AUC	학습 시간	측정
XGBoost (기본)	0.8415	7.1초	실측

XGBoost (HyperOpt 튜닝)	0.8457	—	교재값
LightGBM (기본)	0.8384	1.9초	실측
LightGBM (HyperOpt 튜닝)	0.8446	—	교재값

6.4 머신러닝 활용 포인트

- 불균형 데이터에는 AUC : 불만족 4%인 데이터에서 정확도는 무의미합니다. 문제 성격에 맞는 지표를 먼저 정하세요.
- describe()로 위장된 결측치 찾기 : info() 는 결측치 0건이라고 했지만 describe() 의 min이 -999999였습니다. 3.6장의 피마 데이터와 같은 패턴입니다.
- 익명화된 피쳐도 중요도로 해석 가능 : 이름을 모르더라도 plot_importance 로 어떤 피쳐가 결정적인지는 알 수 있습니다.
- 3단계 데이터 분할 : 학습/검증/테스트로 나뉘야 조기 중단이 정직하게 작동합니다. 테스트 세트를 조기 중단에 쓰면 성능이 부풀려집니다.
- 학습 로그로 과적합 진단 : 학습 AUC와 검증 AUC의 격차가 벌어지는 시점이 과적합의 시작입니다.
- 튜닝은 마지막 몇 % : 기본 모델이 0.8415, 튜닝 후 0.8457로 약 0.5% 향상입니다. 전처리와 피쳐 엔지니어링이 훨씬 큰 영향을 준다는 점을 기억하세요.
- 속도도 성능의 일부 : LightGBM이 AUC는 조금 낮아도 3.7배 빠릅니다. 반복 실험이 필요한 실무에서는 이 차이가 결정적입니다.

⚠ 안정화 버전 참고

- XGBoost 2.0+ : fit() 의 early_stopping_rounds , eval_metric 제거 → 생성자로 이동. 구버전 코드는 TypeError .
- LightGBM 4.0+ : fit() 의 early_stopping_rounds , verbose 제거 → callbacks=[early_stopping(n), log_evaluation(0)] 사용.
- pandas 2.x : df['col'].replace(..., inplace=True) 연쇄 할당은 FutureWarning 발생. 재대입 방식으로 변경하세요.
- HyperOpt 0.3.0 : rstate 에 np.random.default_rng(seed) 를 사용합니다.
- train_test_split 에 stratify=y_labels 를 추가하면 불균형 비율을 정확히 유지할 수 있습니다. 교재 코드에는 빠져 있습니다.